



ShadowKV: KV Cache in Shadows for High-Throughput Long-Context LLM inference

Hanshi Sun^{1,2*}, Li-Wen Chang², Wenlei Bao², Size Zheng², Ningxin Zheng²,

Xin Liu², Harry Dong¹, Yuejie Chi¹, [Beidi Chen](#)¹

¹Carnegie Mellon University ²ByteDance

*Work done during the internship at ByteDance



字节跳动 | Seed

ICML 2025 Spotlight

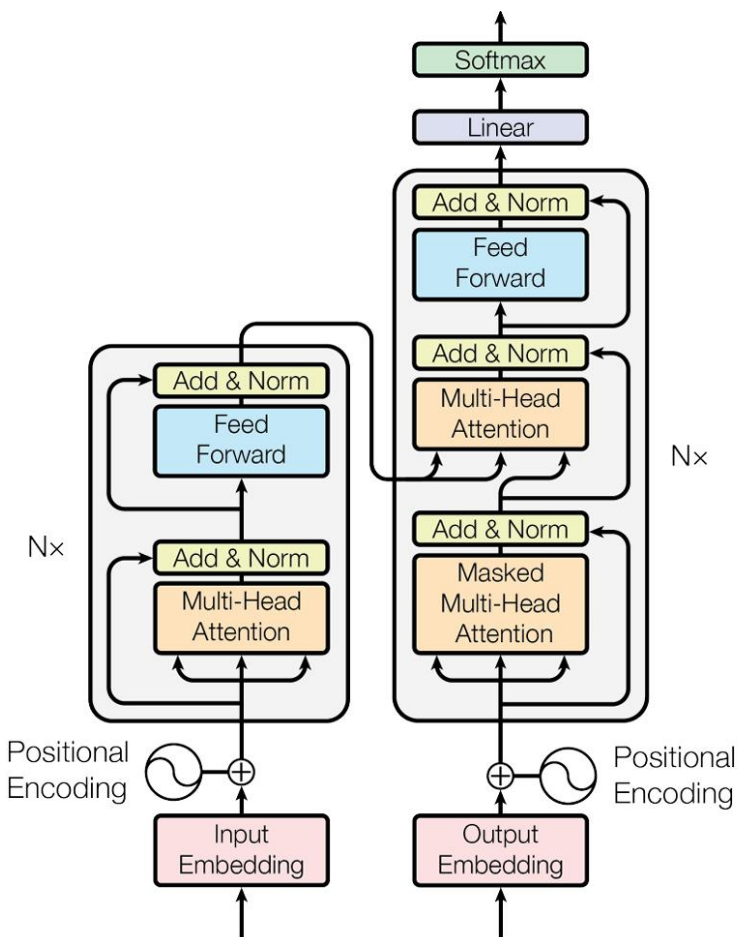
用于高吞吐量、长上下文 LLM 推理的 ShadowKV 缓存机制

<https://bytedance-seed.github.io/ShadowKV/>

小组成员：王锦 王怡雯

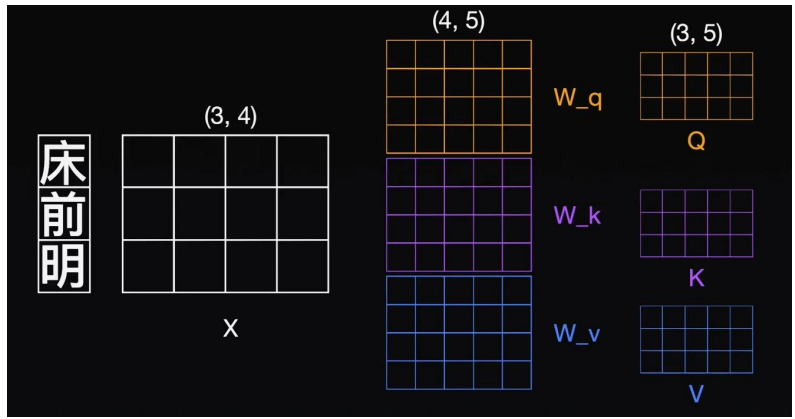
汇报时间：2026.06.29

What is KV cache?



自回归生成下一个 Token 然后拼接

$$Q = XW_Q$$
$$K = XW_K$$
$$V = XW_V$$



计算冗余巨大

生成第 N+1 个 Token 时:
前面 N 个 Token 的 K/V:
又重新算了一遍

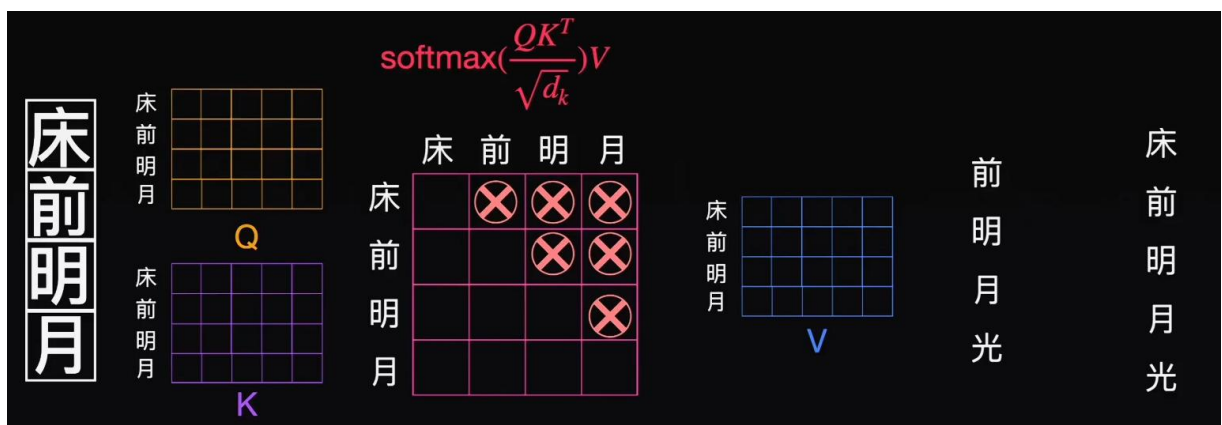
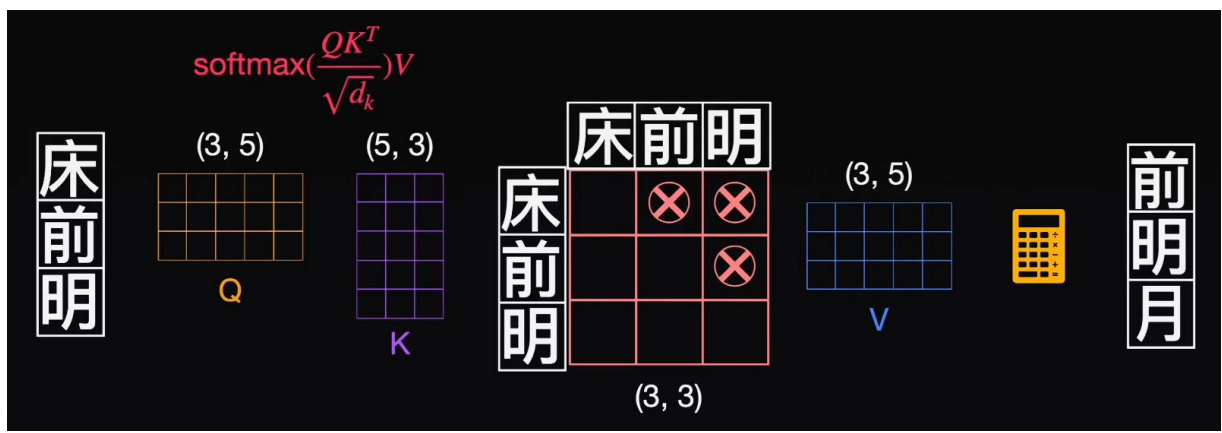
实际上:
只需要将第 N+1 个 Token
的 Query 和 Key 相乘

得到第 N+1 个 Token 与所有
Token 的注意力分数

其余注意力分数全部继承
上一步的计算结果

历史 token 的 K/V 不变
但需重复计算

缓存



Challenge

随着上下文长度扩展至百万级，KV 缓存已成为 LLM 推理的显存瓶颈与带宽瓶颈。

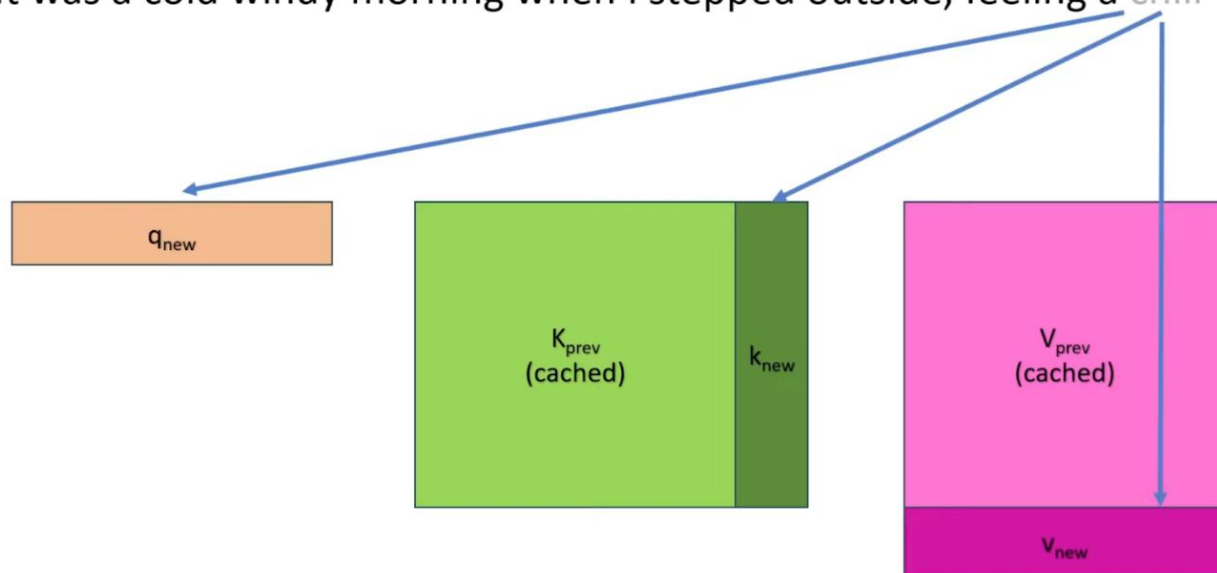
➤ 显存占用随序列长度线性增长

$$M_{KV} = 2 \times L \times T \times H_{KV} \times d_{head} \times b$$

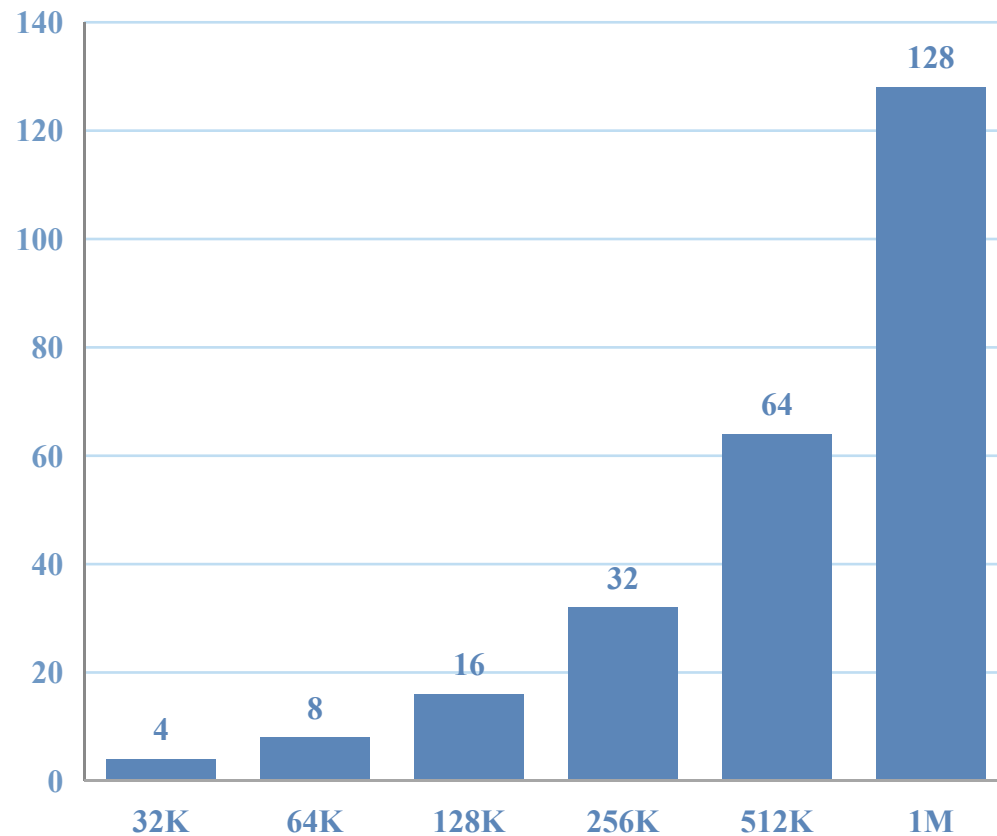
➤ 自回归解码需每步访问完整 KV 缓存，长上下文推理吞吐量很低 访存比计算更贵

空间换时间 多占显存——>少做计算

It was a cold windy morning when I stepped outside, feeling a chill



KV Cache 显存占用随序列长度增长 (Llama-3.1-8B)



Existing Solutions

方法类别	描述	优势	局限性
Sparse Attention	只关注少量重要 Token	减少计算与访存	易遗漏重要 Token (Recall低)
KV Compression	压缩历史 KV 表示	节省 GPU 显存	精度下降
KV Offloading	KV 存储于 CPU/Host Memory	支持超长上下文	CPU-GPU 数据传输瓶颈

如何同时实现低显存、高吞吐和高精度？

ShadowKV 结合稀疏检索、Key 压缩和 Value 卸载三种思想，在保证精度的同时显著降低显存占用并提升吞吐率。

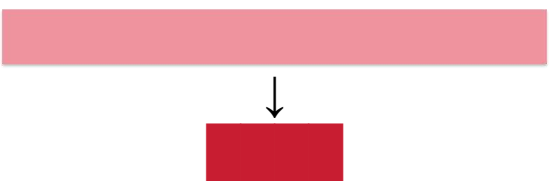
Insight1——Pre-RoPE Keys are Low-Rank

Pre-RoPE Key

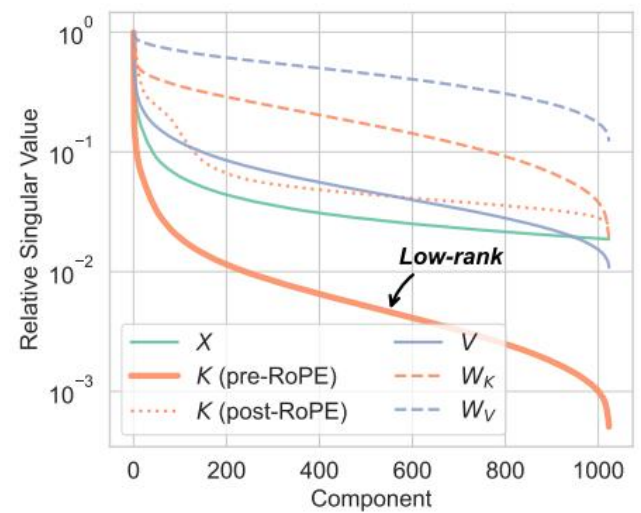
应用RoPE前的原始键向量，分布高度集中，不同位置的键共享相似的低秩子空间

Low-Rank

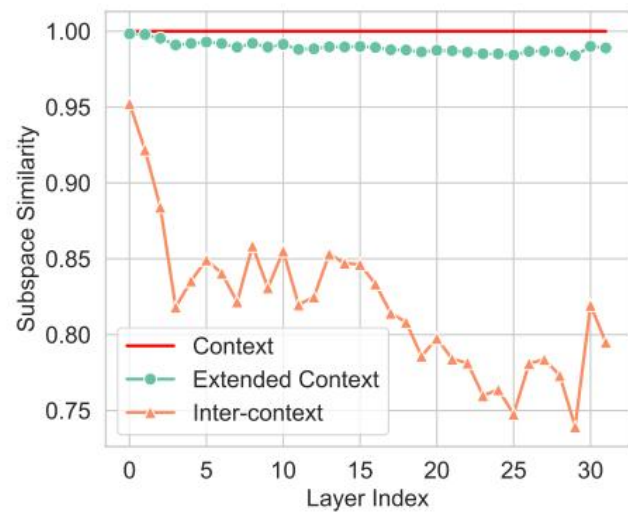
一个高维矩阵里大部分行、列互相线性相关，仅用极少数独立基底就能近似还原整个矩阵，信息高度冗余



① 奇异值谱分析



② 子空间相似性分析



➤ Pre-RoPE Key 具有明显的低秩特性

➤ 同一上下文中的低秩子空间高度一致

离线学习一个共享的投影矩阵，并在推理阶段复用

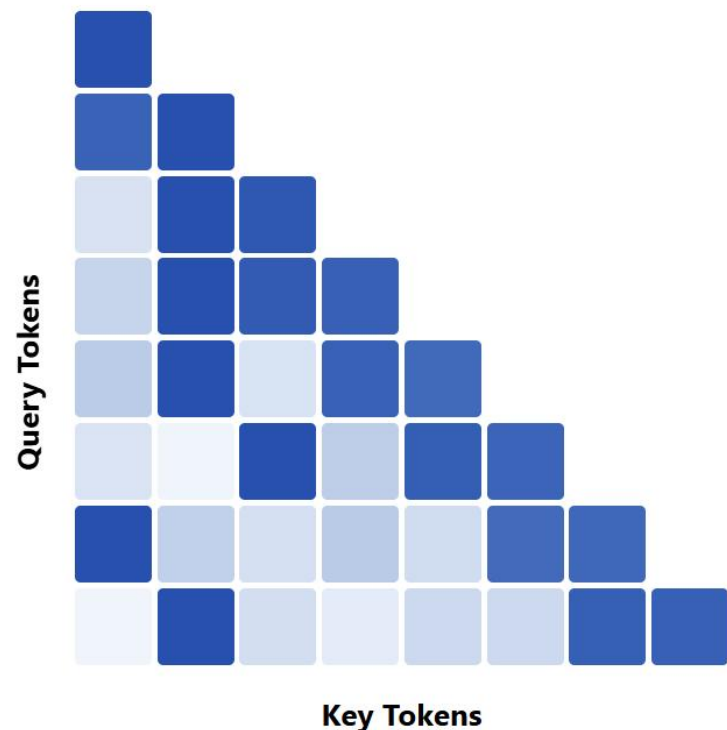
近年来很多大语言模型使用旋转位置编码（RoPE）。RoPE 的核心思想是：不直接把位置编码“加到”输入向量上，而是把位置信息编码进 Q 和 K 的旋转变换中。
对于二维向量对，可以理解为：

$$\begin{bmatrix} x'_1 \\ x'_2 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

奇异值分解
 $SVD(M) = U \Sigma V^T$

Insight2——Not All KV Pairs Are Equally Important

Attention Sparsity



➤ Observation 1:注意力分布高度稀疏

大部分 Attention 权重集中在少量历史 Token 上。

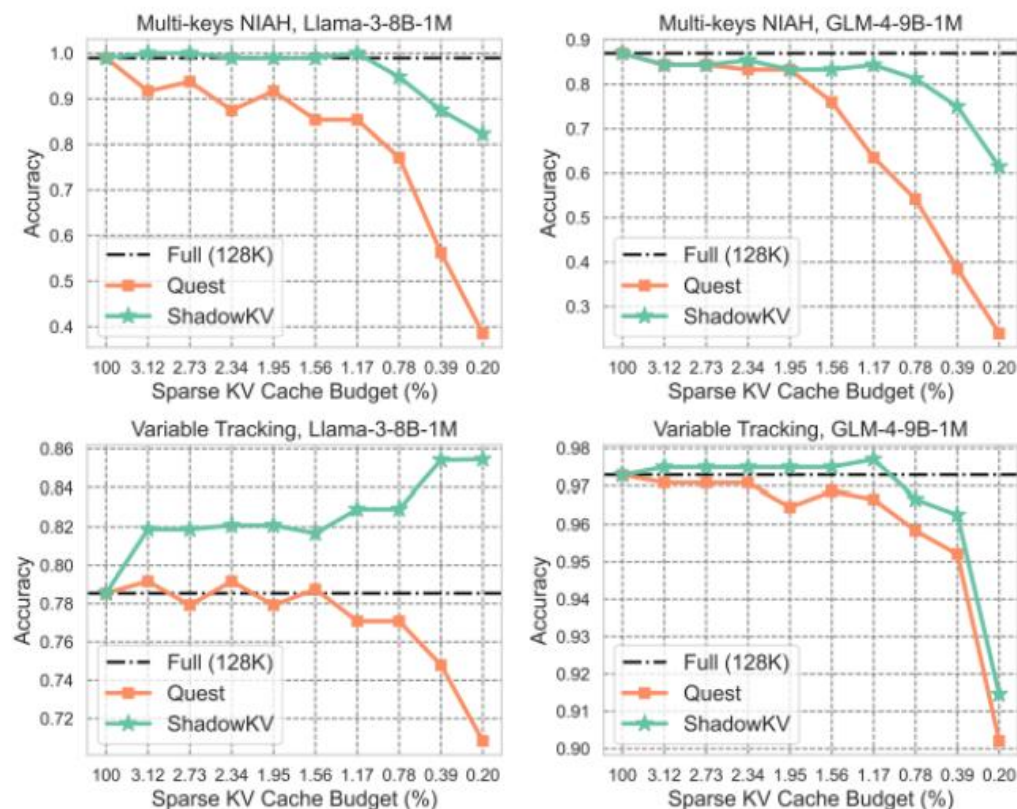
➤ Observation 2:少量 Token 决定模型输出

仅检索少数高相关 Token 即可近似完整 Attention。

➤ Observation 3:无需访问全部 KV Cache

仅恢复 Top-K Token 对应的 KV 即可完成计算

Sparse KV Budget Analysis



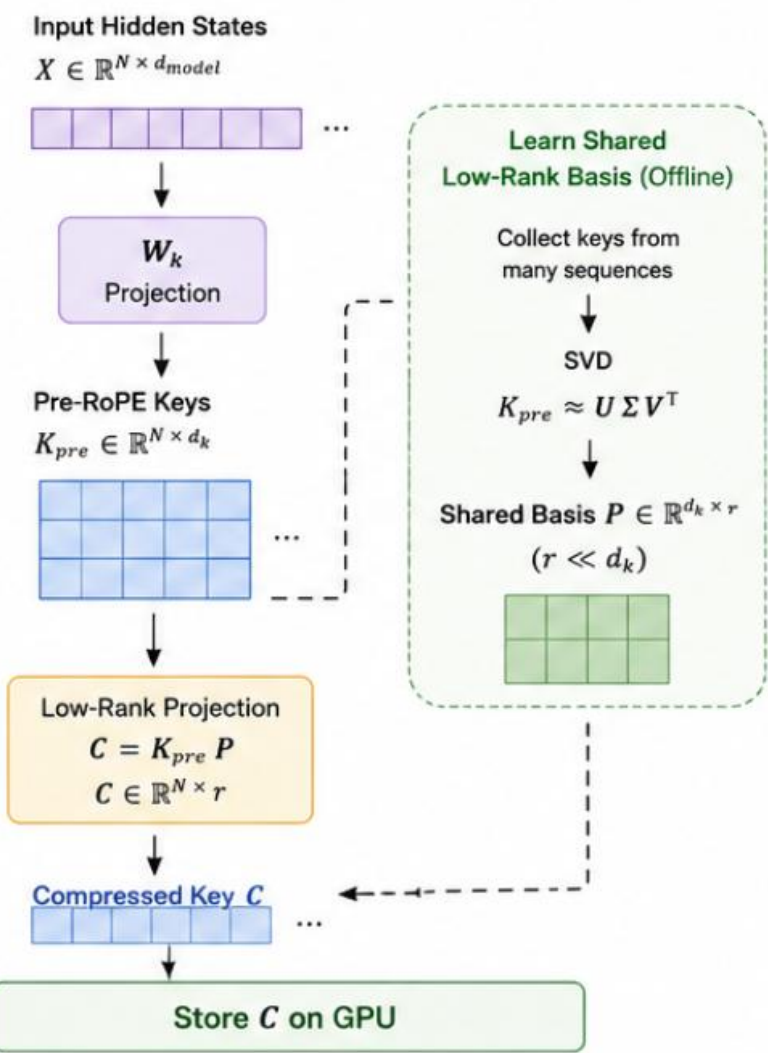
在大多数任务中，仅需 **1.56 %** 的稀疏预算即可保持与全注意力机制相当的准确率

注意力主要集中在少量历史 Token 上，无需访问全部 KV Cache。

Method

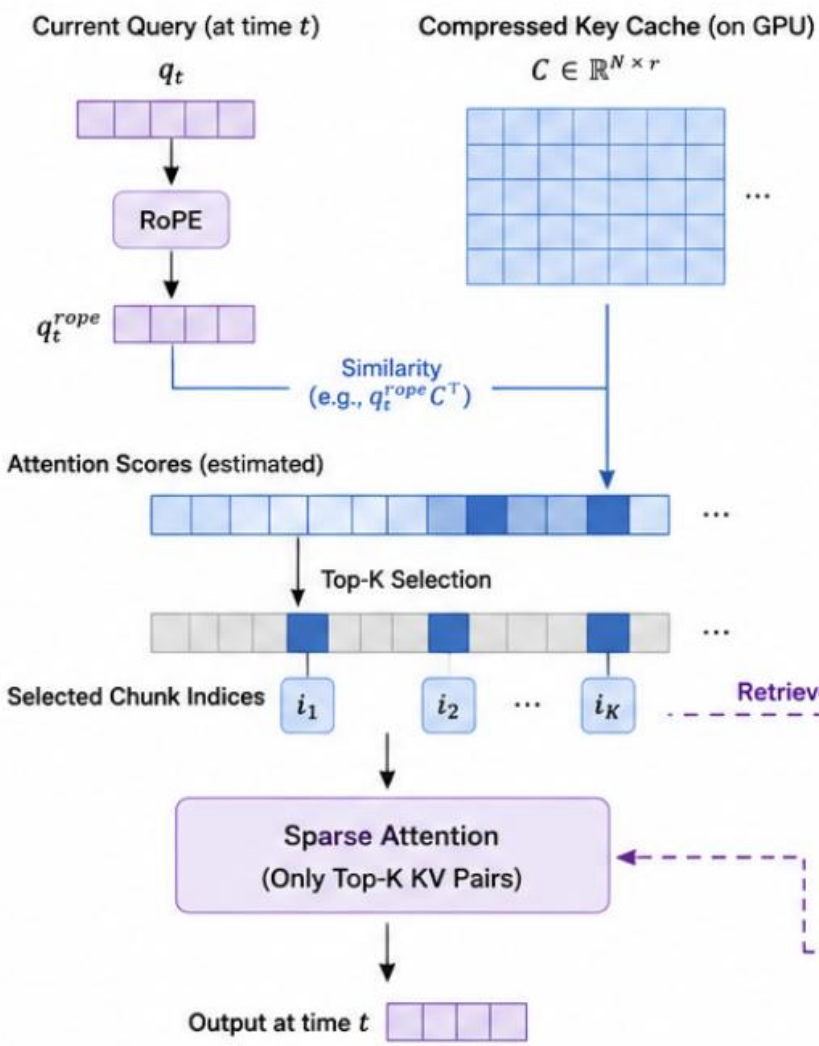
1 Offline Low-Rank Compression

(Prefill Stage / Offline)

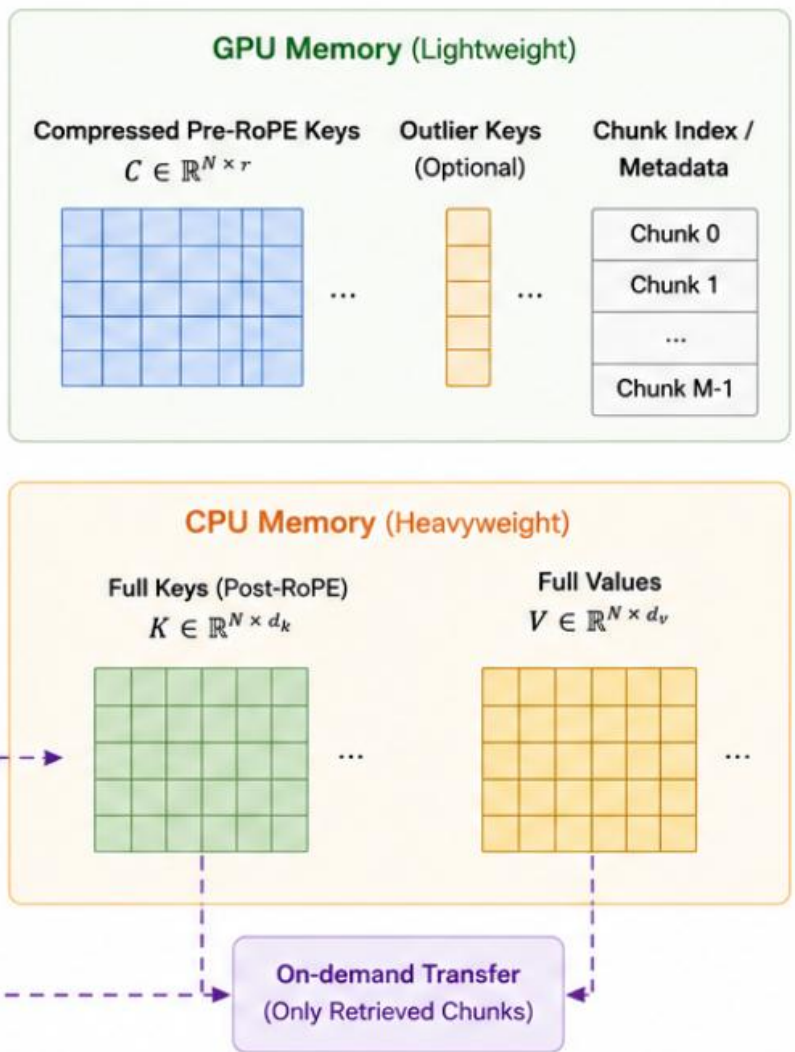


2 Retrieval-based Sparse Attention

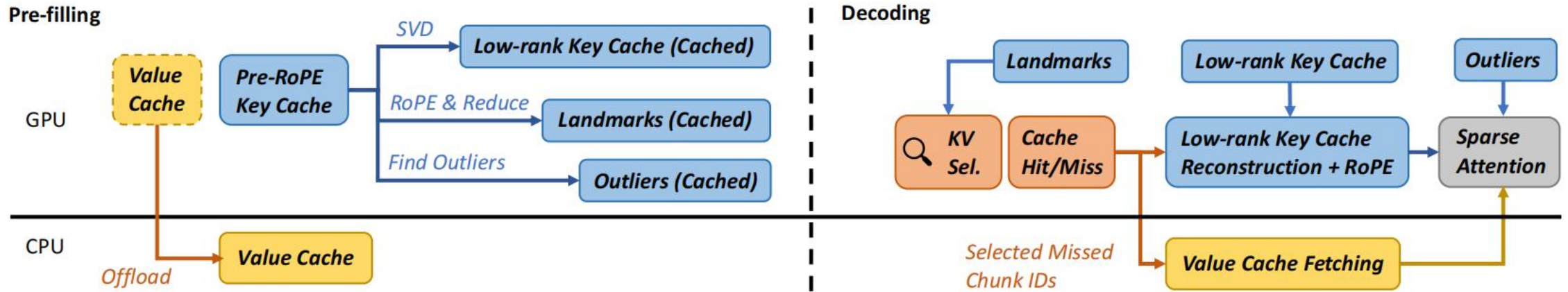
(Decode Stage)



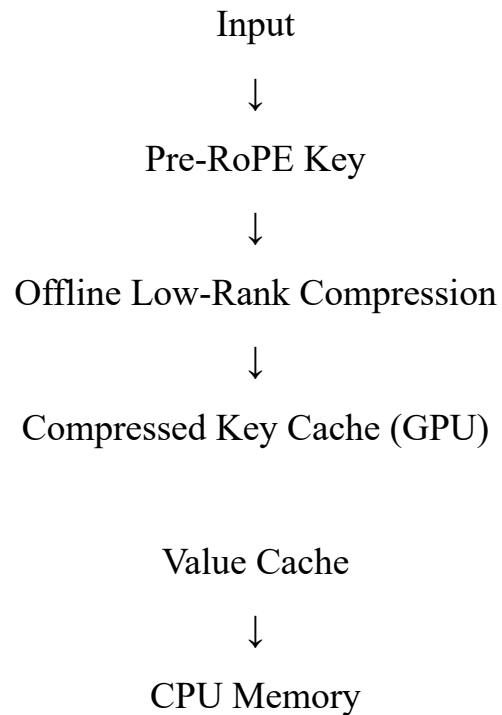
3 Hierarchical Memory Structure



Overall Framework



Prefill Stage:

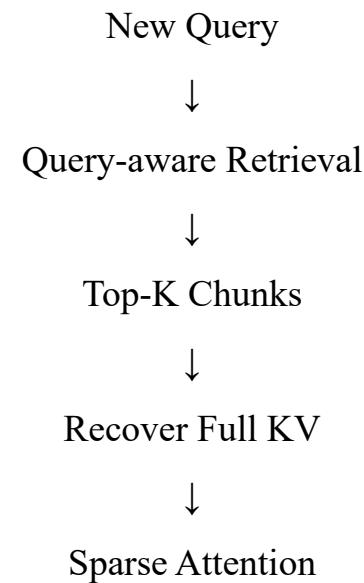


Algorithm 1 SHADOWKV Pre-filling

Input: $K, K^{\text{RoPE}}, V \in \mathbb{R}^{b \times h_{kv} \times s \times d}$, SVD rank r , chunk size c , number of outlier chunks o

- ▷ Store low-rank projection of pre-RoPE key cache
 $A \in \mathbb{R}^{b \times s \times r}, B \in \mathbb{R}^{b \times h_{kv} \times r \times d} \leftarrow \text{SVD}(K)$
- ▷ Segment post-RoPE key cache into chunks and compute the mean of each chunk
 $C \in \mathbb{R}^{b \times h_{kv} \times s/c \times d} \leftarrow \text{Reduce}(K^{\text{RoPE}})$
- ▷ Compute cosine similarity within each chunk
 $S \leftarrow \text{CosineSimilarity}(C, K^{\text{RoPE}})$
- ▷ Find lowest cosine similarity as outliers
 $I \leftarrow \text{ArgTopK}(-\text{Min}(S, \text{dim} = -1), o)$
 $K^{\text{outlier}}, V^{\text{outlier}} \leftarrow \text{Gather}(K^{\text{RoPE}}, V, I)$
- ▷ Offload the rest of values to the CPU and store the non-outlier chunks' mean as landmarks
 $V^{\text{CPU}} \leftarrow V \setminus V^{\text{outlier}}; L \leftarrow C \setminus \text{Gather}(C, I)$

Decode Stage:



Algorithm 2 SHADOWKV Decoding

Input: $A, B, L, V^{\text{CPU}}, Q \in \mathbb{R}^{b \times h_q \times s_q \times d}, K^{\text{outlier}}, V^{\text{outlier}}, K, V \in \mathbb{R}^{b \times h_{kv} \times s_q \times d}$, number of chunks n_c , number of selected chunk budget k

- ▷ Compute chunk attention score
 $P \in \mathbb{R}^{b \times h_q \times s_q \times n_c} \leftarrow \text{MatMul}(Q, L^T)$
 $S \in \mathbb{R}^{b \times h_q \times s_q \times n_c} \leftarrow \text{Softmax}(P/\sqrt{d})$
 $S_1 \in \mathbb{R}^{b \times h_q \times n_c} \leftarrow \text{sum}(S, \text{dim} = -2)$
 $S_2 \in \mathbb{R}^{b \times h_{kv} \times n_c} \leftarrow \text{max}_{kv_group}(S_1)$
- ▷ Select top-k chunks for each KV head
 $I \in \mathbb{R}^{b \times h_{kv} \times k} \leftarrow \text{ArgTopK}(S_2, k)$
- ▷ Gather value cache from the CPU
 $V^{\text{sparse}} \leftarrow \text{Gather}(V^{\text{CPU}}, I)$
 $V \leftarrow [V^{\text{outlier}}; V^{\text{sparse}}; V]$
- ▷ Reconstruct key cache from low-rank projection
 $K^{\text{sparse}} \leftarrow \text{MatMul}(\text{Gather}(A, I), B)$
 $K \leftarrow [K^{\text{outlier}}; \text{RoPE}(K^{\text{sparse}}); K]$

Setup

超参数		对比模型
Parameters	Value	
Sparse KV budget	1.56%	
Chunk size	8	
Rank	160	
Outlier chunks	48	
基准	在每一层、每个 KV Head 中，额外保留 48 个异常块（outlier chunks）在 GPU 上，不参与压缩和 CPU offload。	
Benchmark	Task	GPU：RTX A100（80GB）
RULER	长上下文检索/推理	
LongBench	QA、总结、代码	
NIAH	Needle Retrieval	

Main Accuracy Results

Methods	S1	S2	MK1	MK2	MQ	MV	QA-1	QA-2	VT	FWE	Avg.	NQA	MQA	HQA	MQue	DRead	GRep	SAM	PRetr	LCC	Avg.
Llama-3-8B-1M	100.00	100.00	98.96	98.96	98.96	95.57	75.00	48.96	78.54	71.85	86.68	18.98	41.84	36.79	21.47	31.93	34.18	35.96	81.50	56.07	39.86
Loki	18.75	1.04	2.08	0.00	1.56	0.78	4.17	13.54	26.04	25.35	9.33	2.26	10.19	5.48	3.16	12.17	28.97	7.84	40.52	31.44	15.78
Loki (V)	41.67	6.25	37.50	1.04	8.07	30.73	10.42	19.79	51.67	37.50	24.46	3.20	21.01	12.41	3.86	17.07	31.24	16.23	52.57	38.10	21.74
InfiniGen	100.00	98.96	84.38	53.13	63.28	54.95	65.63	48.96	81.67	50.35	70.13	14.39	31.46	33.63	17.94	26.65	27.38	21.97	74.30	38.58	31.81
InfiniGen (V)	100.00	98.96	96.88	76.04	81.25	77.08	67.71	50.00	81.67	53.47	78.31	17.83	36.08	35.28	19.64	28.39	29.28	28.12	74.85	45.53	35.00
Quest	100.00	100.00	98.96	77.08	97.65	93.49	60.42	50.00	77.08	65.63	82.03	20.13	36.63	35.00	18.14	24.55	27.11	35.63	79.00	53.64	36.65
Quest (V)	100.00	100.00	98.96	85.42	97.92	95.49	70.83	46.88	78.75	65.63	83.99	17.26	39.51	36.78	18.71	26.41	29.49	35.80	79.50	60.05	38.17
SHADOWKV	100.00	100.00	97.92	98.96	96.88	95.83	72.92	52.08	81.67	72.57	86.88	17.17	39.73	38.29	21.08	31.77	31.62	35.87	80.00	63.93	39.94
GLM-4-9B-1M	100.00	100.00	94.79	87.50	99.74	93.75	67.71	55.21	97.29	72.22	86.82	25.44	51.09	58.67	39.61	32.04	29.97	40.31	99.00	58.02	48.24
Loki	71.88	27.08	22.92	2.08	9.90	11.46	28.13	27.08	31.04	54.17	28.57	5.82	30.60	22.73	9.20	30.09	30.35	22.70	98.92	40.77	32.35
Loki (V)	96.88	55.21	56.25	18.75	51.04	50.52	45.83	39.58	72.71	59.72	54.65	10.89	44.97	45.44	23.51	32.07	30.56	35.34	99.50	50.27	41.39
InfiniGen	100.00	93.75	82.29	0.00	79.43	60.16	57.29	53.13	92.71	57.29	67.60	23.67	46.31	55.69	33.91	27.49	25.44	33.48	91.83	36.96	41.64
InfiniGen (V)	100.00	96.88	87.50	7.29	95.31	75.26	56.25	54.17	95.63	60.76	72.91	25.63	48.44	57.23	36.94	29.77	26.67	36.64	93.58	46.69	44.62
Quest	100.00	95.83	90.62	54.17	94.01	76.30	55.21	52.08	95.83	64.58	77.86	23.81	44.53	56.41	35.49	23.54	21.73	37.39	87.00	43.80	41.52
Quest (V)	100.00	96.88	93.75	72.92	95.83	83.07	56.25	53.13	96.88	65.97	81.47	26.00	46.32	57.54	36.42	24.58	24.52	37.71	93.50	46.52	43.68
SHADOWKV	100.00	100.00	95.83	83.33	98.70	87.76	69.79	55.21	97.50	68.06	85.62	26.50	51.31	59.09	38.87	32.92	28.54	38.70	96.50	58.55	47.89
Llama-3.1-8B	100.00	100.00	98.96	91.67	98.96	95.31	82.29	47.92	68.96	71.18	85.53	31.56	55.10	57.65	29.46	35.26	34.45	29.84	100.00	67.31	48.96
Loki	68.75	32.29	32.29	20.83	42.71	28.65	41.67	33.33	24.79	29.86	35.52	2.31	18.89	10.64	5.47	19.30	31.16	15.91	94.88	44.60	27.02
Loki (V)	95.83	36.46	57.29	62.50	77.86	70.83	69.79	39.58	35.21	37.50	58.29	3.93	38.59	22.85	12.96	27.43	32.22	26.43	98.25	56.11	35.42
InfiniGen	100.00	77.08	78.13	13.54	58.07	47.40	65.63	41.67	60.83	50.35	59.27	27.23	52.72	53.89	26.81	27.72	29.61	24.42	98.93	56.89	44.25
InfiniGen (V)	100.00	88.54	87.50	26.04	79.43	77.08	72.92	43.75	57.08	55.21	68.76	29.73	53.47	55.11	28.72	28.55	31.42	26.76	99.17	62.66	46.18
Quest	100.00	98.96	97.92	34.38	93.49	88.54	70.83	44.79	65.63	68.40	76.29	29.70	49.04	53.96	27.18	27.16	30.43	29.85	98.50	57.35	44.80
Quest (V)	100.00	98.96	98.96	56.25	95.83	90.63	76.04	46.88	66.25	67.36	79.72	30.02	53.97	56.39	27.06	29.06	31.65	30.23	99.00	63.89	46.81
SHADOWKV	100.00	100.00	100.00	83.33	97.92	92.19	81.25	48.96	67.08	64.93	83.57	30.93	55.20	57.32	29.13	31.85	32.79	30.40	99.50	66.03	48.13

ShadowKV 在仅使用**1.56 %** KV Cache 的情况下，仍然保持了与 Full Attention 几乎一致的**精度与长上下文检索能力**。

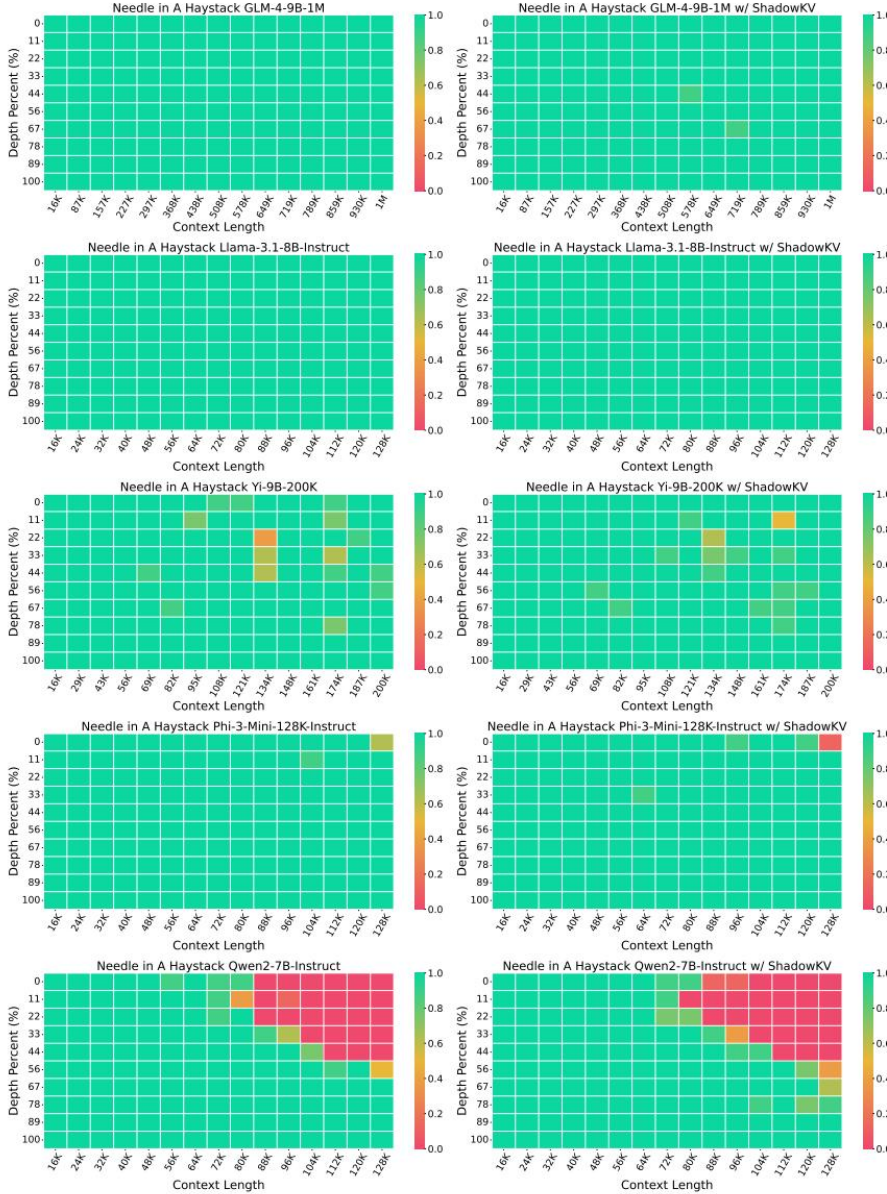


Figure 11 Needle In A Haystack [20] results using GLM-4-9B-1M [12], Llama-3.1-8B-Instruct [31], Yi-9B-200K [3], Phi-3-Mini-128K [1], and Qwen2-7B-128K [54].

Efficiency

Table 3 Generation throughput (tokens/s) on an A100. The gray text in brackets denotes batch size.

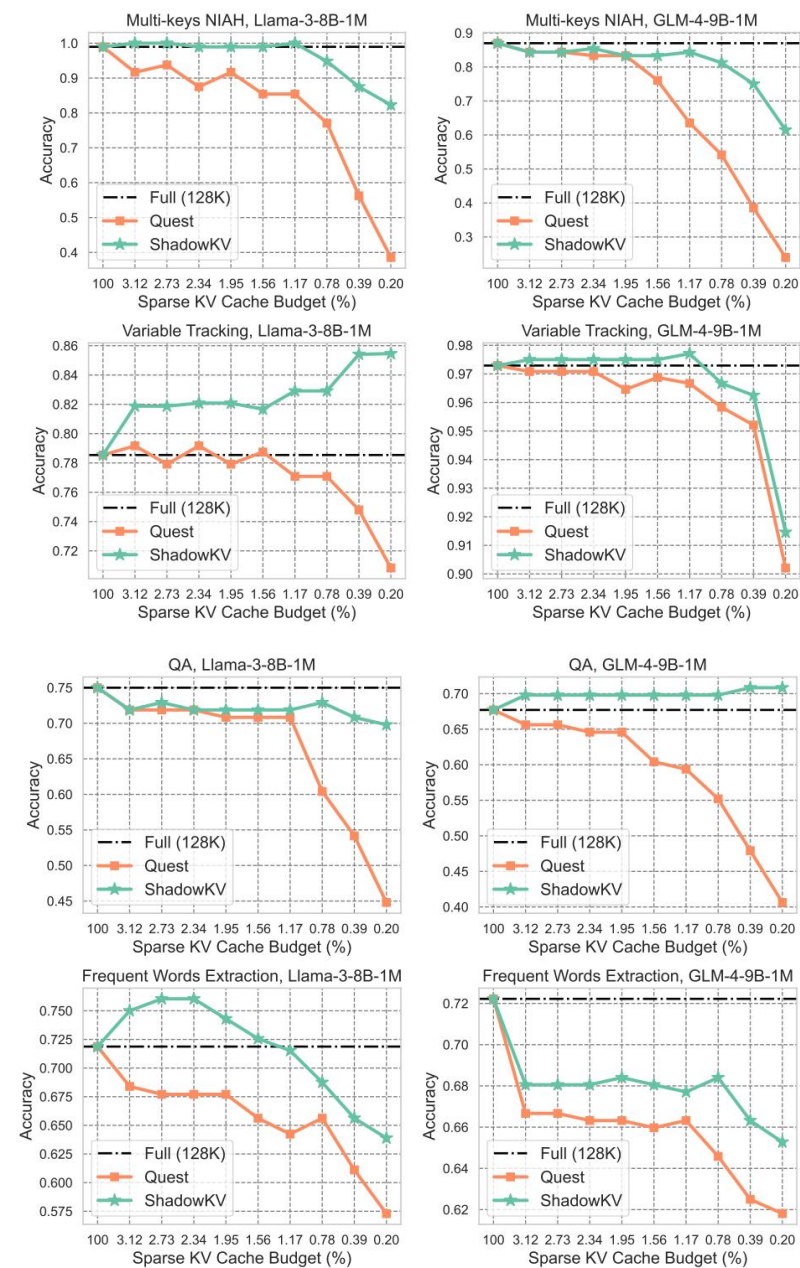
Model	Context	Full Attn	SHADOWKV	Gain	Full Attn (Inf)
Llama-3-8B-1M (8 KV heads)	60K	160.62 (8)	455.14 (48)	2.83×	168.72 (48) / 273.07 (Inf)
	122K	80.77 (4)	239.51 (24)	2.97×	83.05 (24) / 134.30 (Inf)
	244K	40.37 (2)	119.01 (12)	2.95×	52.00 (12) / 67.15 (Inf)
Llama-3.1-8B (8 KV heads)	60K	160.93 (8)	472.77 (48)	2.94×	168.72 (48) / 273.07 (Inf)
	122K	80.78 (4)	245.90 (24)	3.04×	83.05 (24) / 134.30 (Inf)
GLM-4-9B-1M (4 KV heads)	60K	241.05 (12)	615.89 (50)	2.56×	266.24 (50) / 436.91 (Inf)
	122K	122.67 (6)	293.40 (25)	2.39×	158.83 (25) / 214.87 (Inf)
	244K	61.13 (3)	136.51 (12)	2.23×	78.84 (12) / 107.44 (Inf)
Yi-9B-200K (4 KV heads)	60K	204.81 (10)	544.36 (42)	2.66×	271.21 (42) / 364.09 (Inf)
	122K	101.44 (5)	260.03 (21)	2.56×	133.53 (21) / 179.06 (Inf)
	244K	46.74 (2)	118.55 (10)	2.54×	65.79 (10) / 89.53 (Inf)

Table 4 Generation throughput (tokens/s) under varying batch sizes and sequence lengths on Llama-3-8B-1M.

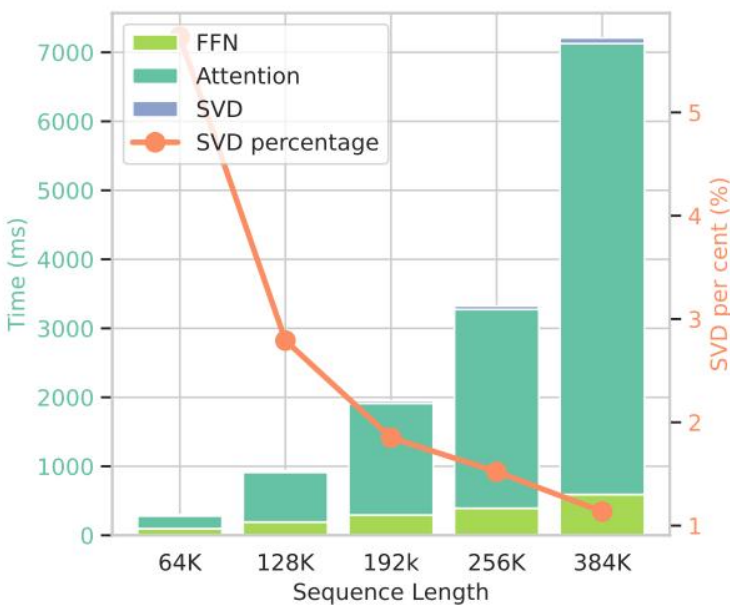
Context	2	3	4	5	6	8	12	16	24	32	48
Full KV											
60K	89.19	111.44	126.73	142.62	147.40	160.62	OOM	OOM	OOM	OOM	OOM
122K	65.12	75.16	80.77	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM
244K	40.37	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM
488K	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM
ShadowKV											
60K	89.69	126.61	159.41	184.92	205.41	244.20	306.09	346.34	399.65	428.63	455.14
122K	65.61	94.28	115.01	132.23	143.77	166.72	196.73	217.24	239.51	OOM	OOM
244K	48.39	65.95	78.92	87.83	94.07	104.73	119.01	OOM	OOM	OOM	OOM
488K	29.82	41.01	47.13	50.85	53.46	OOM	OOM	OOM	OOM	OOM	OOM

显著提升吞吐量 (X3)
支持更大的 Batch(X6)

随着上下文长度增加
Full Attention 很快 OOM
ShadowKV 支持大规模
Batch 推理
展现出优异的可扩展性



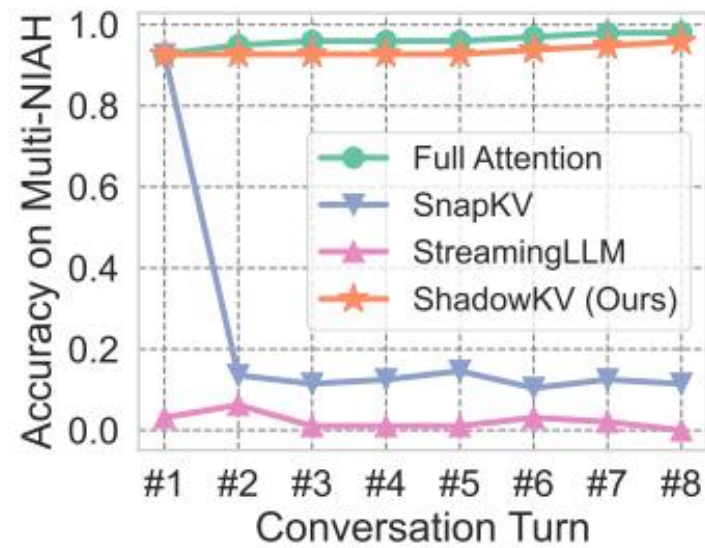
Ablation



➤ SVD 开销分析

随着上下文长度增加，Attention 开销是主要瓶颈，低秩分解（SVD）的开销始终占比较小

证明 ShadowKV 的低秩重建成本可以忽略



➤ 多轮对话实验

在多轮对话场景下，ShadowKV 始终保持与 Full Attention 接近的性能
基于固定保留策略的方法（如 SnapKV、StreamingLLM）会随着轮次增加而严重退化。

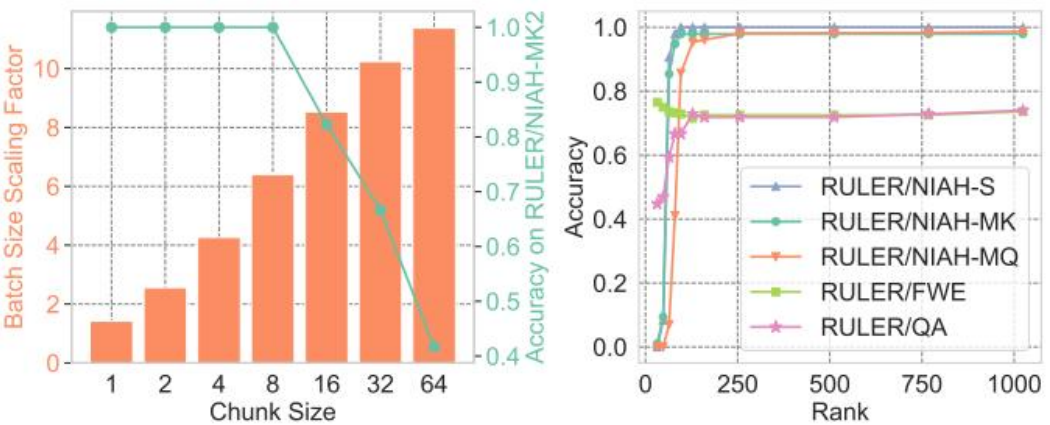


Figure 9 Left: Impact of chunk size on batch size and accuracy. Right: Accuracy trends across different ranks.

➤ Chunk Size 与 Rank 消融

Chunk Size = 8

增大 Chunk Size 能显著提升可支持的 Batch Size，但过大的 Chunk 会导致检索精度明显下降

Rank = 160

在精度与效率之间取得良好平衡

Reproduction

复现环境

项目	配置
GPU	NVIDIA RTX PRO 6000
GPU 显存	96GB
torch	2.11.0+cu128
CUDA toolkit	CUDA 12.8

模型与数据

项目	配置
model	Llama-3-8B-Instruct-Gradient-1048k
dataset	NIAH

打开的编辑器

shadowkv_core_reproduction... U

{ validation.jsonl data/ruler/data/llam...

SHADOWKV [SSH: CONNECT.WESTD.SEETAC...

data

niah

ruler

__pycache__

data

llama-3

4096

32768

niah_multikey_1

{ validation.jsonl

niah_single_1

{ validation.jsonl

qa_1

data > ruler > data > llama-3 > 32768 > niah_single_1 > { validation.jsonl

1 {"index": 0, "input": "<|begin_of_text|><|start_header_id|>system<|end_header_id|>You is a helpful assistant<|eot

2 {"index": 1, "input": "<|begin_of_text|><|start_header_id|>system<|end_header_id|>You is a helpful assistant<|eot

3 {"index": 2, "input": "<|begin_of_text|><|start_header_id|>system<|end_header_id|>You is a helpful assistant<|eot

4 {"index": 3, "input": "<|begin_of_text|><|start_header_id|>system<|end_header_id|>You is a helpful assistant<|eot

5 {"index": 4, "input": "<|begin_of_text|><|start_header_id|>system<|end_header_id|>You is a helpful assistant<|eot

6 {"index": 5, "input": "<|begin_of_text|><|start_header_id|>system<|end_header_id|>You is a helpful assistant<|eot

7 {"index": 6, "input": "<|begin_of_text|><|start_header_id|>system<|end_header_id|>You is a helpful assistant<|eot

8 {"index": 7, "input": "<|begin_of_text|><|start_header_id|>system<|end_header_id|>You is a helpful assistant<|eot

9 {"index": 8, "input": "<|begin_of_text|><|start_header_id|>system<|end_header_id|>You is a helpful assistant<|eot

10 {"index": 9, "input": "<|begin_of_text|><|start_header_id|>system<|end_header_id|>You is a helpful assistant<|eot

11 {"index": 10, "input": "<|begin_of_text|><|start_header_id|>system<|end_header_id|>You is a helpful assistant<|eot

12 {"index": 11, "input": "<|begin_of_text|><|start_header_id|>system<|end_header_id|>You is a helpful assistant<|eot

13 {"index": 12, "input": "<|begin_of_text|><|start_header_id|>system<|end_header_id|>You is a helpful assistant<|eot

14 {"index": 13, "input": "<|begin_of_text|><|start_header_id|>system<|end_header_id|>You is a helpful assistant<|eot

15 {"index": 14, "input": "<|begin_of_text|><|start_header_id|>system<|end_header_id|>You is a helpful assistant<|eot

16 {"index": 15, "input": "<|begin_of_text|><|start_header_id|>system<|end_header_id|>You is a helpful assistant<|eot

Reproduction

精度任务 1: ruler/niah_single_1

任务目的：验证模型能否在 **128K** 上下文中找到**单个**隐藏数字。

输入形式：长文档中插入一句类似“special magic number is ...”的关键信息，末尾提问该数字是什么。

评价方式：模型输出数字与 ground truth 匹配则计为正确。

验证长上下文单目标召回。

Processes:						
GPU	GI	CI	PID	Type	Process name	GPU Memory Usage
	ID	ID				
0	N/A	N/A	2083	C	...-tmp/envs/ShadowKV/bin/python	34802MiB

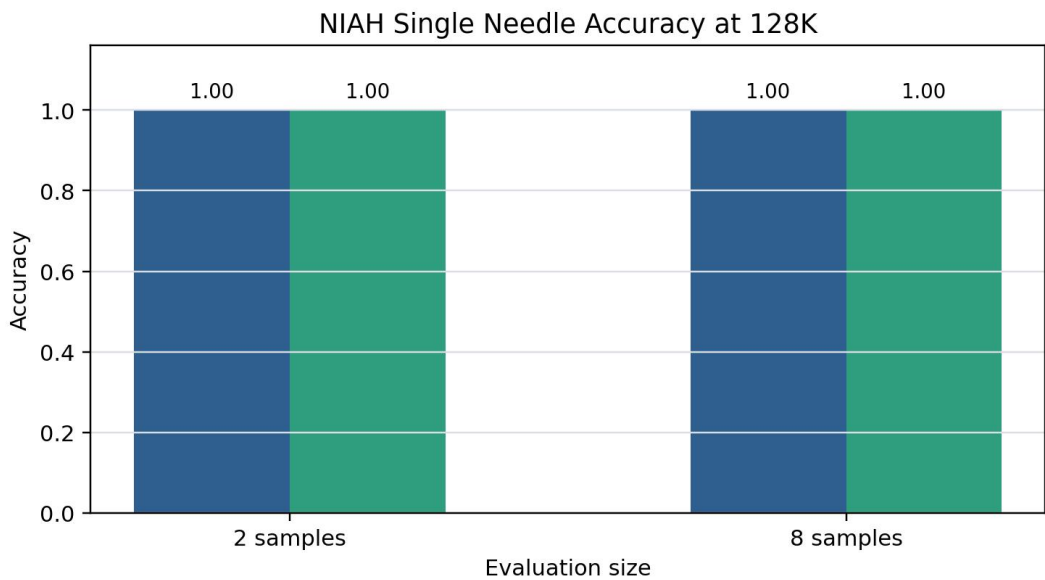
精度任务 2: ruler/niah_multikey_1

任务目的：验证模型能否在 **128K** 上下文中处理**多 key** 检索场景。

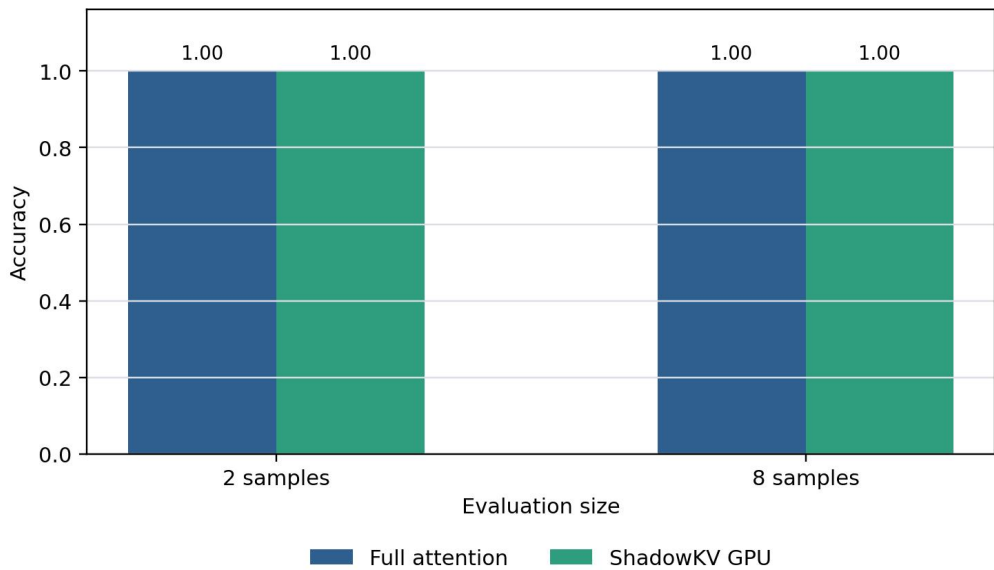
输入形式：上下文中存在多个 key/value 风格的隐藏信息，问题要求回答指定 key 对应的数字。

评价方式：模型输出指定 key 对应数字则计为正确。

更难，不仅要找到数字，还要**避免召回错误** key 对应的数字。

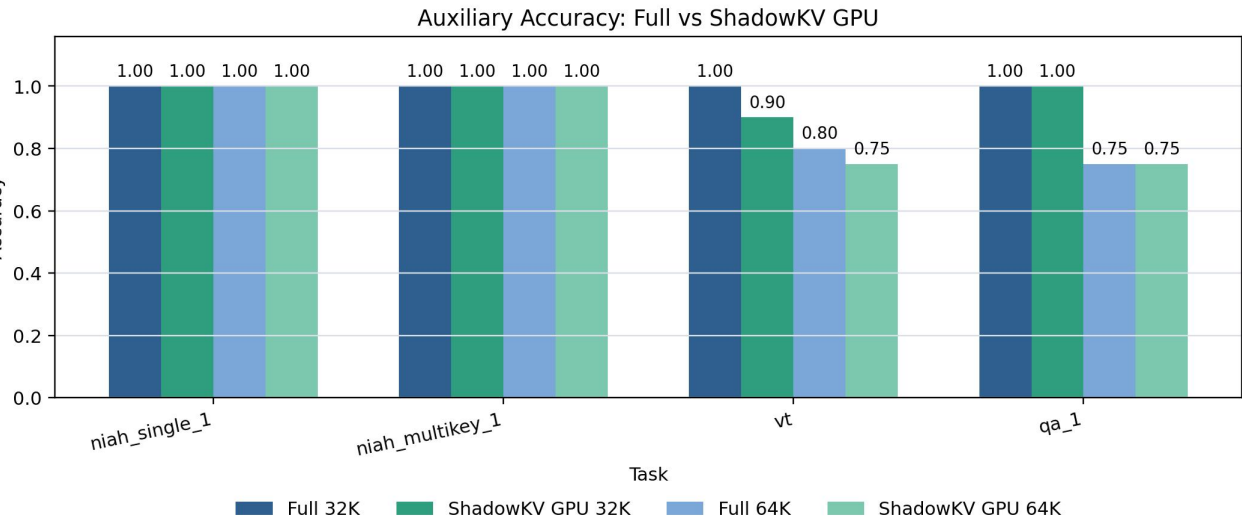


25.8%, 25.8%, 40.8%, 21.4%, 53.4%, 27.6%, 13.4%, 81.9%



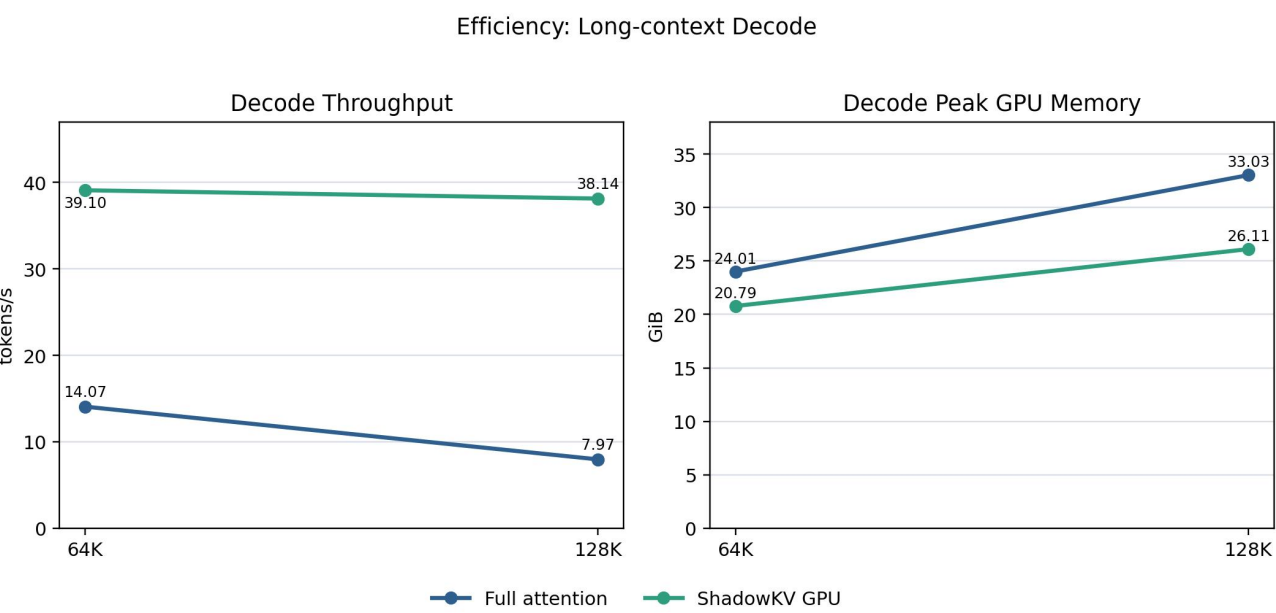
Reproduction

精度任务 3：32K/64K 辅助



在 32K/64K 辅助任务上，ShadowKV 与 full attention 在两个 NIAH 任务中均保持 1.0；
vt 和 qa_1 在小样本下存在波动，但 64K 时 ShadowKV 与 full attention 的结果基本接近。

效率任务 1：ruler/niah_multikey_1



128K 下 shadowkv相比 full attention 的 decode 加速为 **4.78x**
128K 下 shadowkv的 decode 峰值显存为 **26.11 GiB**，低于 full attention 的 **33.03 GiB**。

Conclusion

- 提出 ShadowKV, 通过低秩重建 Key, 实现高效的 KV Cache 压缩。
- 显著减少 CPU-GPU 通信开销, 提升长上下文推理效率。
- 在几乎无精度损失的情况下, 实现最高 $3.04\times$ 吞吐提升和 $6\times$ Batch 扩展能力。

感谢聆听

